

生成-评估优化循环 · Eval Mechanism

Goal 实现质量评估 → 失败分类 → Replan 反馈 → 代码优化迭代

入口

👤 用户请求

自然语言任务描述
e.g. 构建股票交易系统

需求解析

📄 RequirementsAgent

结构化需求分析
生成 SpecSnapshot
输出 requirement_ids

接口设计

🏗️ ArchitectAgent

跨 Goal 接口契约设计
解决模块边界冲突
owned_paths 分配

执行规划

📅 PlannerAgent

生成并行 DAG 计划
每个 Goal 独立 worktree
priority: blocking / advisory

GOAL 拆解

⚡ Goal 并行分发

queueGoalRun()
每个 Goal 独立 worktree
并发执行无共享状态

Goal A **Goal B** **Goal N**

最终合并

✅ Merge to Main

cherry-pick / patch apply
清理 worktree 分支
DeliveryAgent 验收后触发

REPLAN 反馈路径

① **Evaluator** 产出 replan_guidance
② **Planner** 根据 avoid_approaches 重规划
③ **Build Agent** 在新 worktree 重新实现
④ **Evaluator** 再次验收, 直至 accepted

GOAL 执行层 · 每个 GOAL 独立 WORKTREE

🔧 Build Agent (AI Coding Agent)

在隔离的 git worktree 中实现 Goal · owned_paths 边界约束 · 读写文件 + 执行命令

owned_paths 隔离 **独立 git branch** **cwd = worktreeDir**

代码快照

📦 PipelineDelivery · Diff Snapshot

· changedFiles[] · diffs { file, diff } · summary · worktree branch

验收层 · PER-GOAL AUTONOMOUS JUDGE

🔍 EvaluatorAgent · per-goal.ts

自主评估, 无预定义检查项 · 读代码 → 推断检查 → 执行 → 产出裁决

评估执行序列 · MAX_STEPS=30 · TIMEOUT=5MIN

① **读取源码** — 逐文件检查 changedFiles, 理解实现意图与数据流

② **验证 owned_paths** — 检测是否修改了 Goal 边界外的文件, 违规即 REJECT

③ **推断检查策略** — 根据代码类型自动选择验证手段:
React/Vue → build 检查 · 后端服务 → 接口测试 · 库代码 → 单元测试

④ **run_command 执行检查** — 运行 build / test / lint / verify_cmd

⑤ **done_definition 对照** — 逐条核查 Goal 的完成定义, 收集具体 evidence (文件路径 + 行号 + 断言结果)

⑥ **Goal Judge (judge.txt)** — 严格基于证据评分, 拒绝“测试通过”等模糊表述

⑦ **输出结构化裁决** — EvalVerdict: verdict + confidence + evidence[] + failureClass + replan_guidance

自动推断检查项

🔧 build
bun run build
TypeScript 编译无错误

🚀 test
bun:test 单元测试
引用路径 + 断言输出

🔍 lint
biome check
代码规范与类型安全

✅ verify_cmd
自定义验证命令
接口契约 + E2E 检查

⚠️ DEADLOCK 检测

- 同一测试连续失败 3+ 次且错误完全相同
- Agent 在两种方案间交替但无进展
- 修改仅为表面变化, 未解决根本问题

→ deadlock=true 时直接 REJECT, 不再等待 max_steps

裁决输出

🔍 EvalVerdict · GoalJudgment

EvaluatorAnalysis 结构化输出 · 驱动后续流程分支

✅ ACCEPTED
所有 CRITERIA 满足
文件级证据完整

❌ REJECTED
CRITERIA 未满足
触发 REPLAN 流程

? INCONCLUSIVE
证据不足
保守判定为失败

GOALASSESSMENT

goal_index: number
status: passed | failed | inconclusive
evidence: string[] 具体行号 + 断言
reasoning: string

EVALUATORANALYSIS

verdict: enum
classification: FailureClass
goal_statuses: GoalAssessment[]
replan_guidance: object | null

REPLAN

失败处理

🔄 Replan + 重建 Worktree

根据 replan_guidance 生成新 GoalContract
新 worktree 分支, 历史 diff 清空
Build Agent 采用新策略重新实现

交付验证

📦 DeliveryAgent

runtime 启动验证
集成 bug 修复 (不改架构)
最终 verdict → 触发 Merge

REPLANGUIDANCE

🔍 root_cause
技术层面真实原因
e.g. “WebSocket 消息类型”

💥 what_failed
具体失败的交付部分
e.g. “实时行情 WebSc”

🔧 suggested_strategy
下次尝试的不同方向
e.g. “改用 JSON.string”

🚫 avoid_approaches
已尝试且失败的方案
Build Agent 必须跳过

REPLANGUIDANCE 流转链

EvaluatorAgent
↓
I replan_guidance
PlannerAgent
↓
I new GoalContract (avoid...
Build Agent
↓
I new worktree + 新策略
EvaluatorAgent (再次验收)

JUDGE · 核心评判规则

1 **仅凭证据判断**
禁止采纳 Agent 自我

2 **保守原则**
存疑时判为未达成

3 **调查后再判**
证据模糊时必须用工具

4 **具体 nextStep**
必须给出精确文件路径

5 **Deadlock 检测**
重复错误 / 无效循环

GOOD evidence:

“read_file src/ws.ts:42 — bi
缺少 replacer, 导致 JSON

BAD evidence (拒绝):

“Tests pass” · “代码看起来
“功能应该可以工作”

